

Vulkan 特性

1 文档说明

本文档主要用于介绍 Vulkan 特性。一方面记录个人对 Vulkan 的学习，另一方面便于在工作和个人研究中查阅。

参考：

Vulkan 教程

2 Dynamic Uniform Buffer

2.1 背景

对于普通的 uniform buffer：

- 每个 uniform buffer 数据需要绑定一个 descriptor set
- 如果有 100 个物体，每个物体都有自己的 MVP 矩阵，那么有两种做法：
 - (1) 创建 100 个 descriptor set，每个绑定一个 uniform buffer
 - (2) 每次绘制前更新同一个 uniform buffer

这会带来明显的性能问题，所以我们引入 dynamic uniform buffer，可以做到：

- 只创建一个 descriptor set

- 在一个 uniform buffer 里存储所有物体需要的数据
- 通过 `vkCmdBindDescriptorSets` 的 `dynamic offset` 参数，在绘制时指定偏移量

2.2 优点

- (1) 减少 descriptor set 数量：1 个 descriptor set 可以服务 N 个对象
- (2) 避免频繁更新 uniform buffer：数据一次性写入 buffer，在绘制时通过 `dynamic offset` 指定偏移量
- (3) 提升 CPU 性能：减少 API 调用和驱动开销
- (4) 对多线程友好：每个线程处理不同偏移的数据，同时更新 buffer

2.3 使用限制

(1) `dynamicOffset` 必须是设备属性 `minUniformBufferOffsetAlignment` 的倍数

```

1 size_t minUboAlignment =
2     vulkanDevice->properties.limits.minUniformBufferOffsetAlignment;
3 dynamicAlignment = sizeof(glm::mat4);
4 if (minUboAlignment > 0) {
5     dynamicAlignment =
6         (dynamicAlignment + minUboAlignment - 1) & ~(minUboAlignment - 1);
7 }

```

(2) 受 `maxDescriptorSetUniformBufferDynamic` 限制，位于结构体 `VkPhysicalDeviceLimits` 中，它表示一个 descriptor set 中最多可以绑定多少个 dynamic uniform buffer。

(3) 仅适用于 uniform buffer 和 storage buffer

2.4 使用场景

- 多物体的 MVP 矩阵
- 多光源渲染

- 骨骼动画的骨骼变换矩阵

3 Push Constants

3.1 背景

对于小规模、频繁变化的数据，使用 uniform buffer 的性价比不高。Vulkan 提出 push constants，可以更高效地传递数据。

3.2 优点

- 不需要分配 descriptor set 和 uniform buffer（不用分配显存）
- 直接通过 command buffer 更新数据，降低性能开销
- 适用于高频更新
- 对应 pipeline layout，可以在多个 shader stage 共享

3.3 使用限制

- (1) 最大容量限制：一般为 128 字节，可以通过 `VkPhysicalDeviceLimits::maxPushConstantsSize` 来查询
- (2) 必须与 pipeline layout 匹配
- (3) 字节对齐（与 uniform buffer 相同）

3.4 使用场景

- 简单材质参数（动画过程中更新）
- 状态切换（少量 bool 值或浮点数的变化）

4 Specialization Constants

4.1 背景

如果要根据不同配置（如光照数量、是否使用阴影等）使用不同着色器逻辑，一般有以下两种方法：

- 创建多个着色器
- 使用 `#define`，然后多次编译

但是这会导致：

- 需要创建大量着色器
- 编译管理复杂
- 无法在运行时灵活切换

这种情况下可以考虑使用 Specialization Constants，可以实现：

- 只创建一个着色器
- 在创建管线时对常量进行动态“特化”
- 驱动/Vulkan 会进行优化（如死代码消除）

其本质是编译期常量，在管线创建时确定，但是由 CPU 在运行时提供。

4.2 优点

- 减少着色器数量：一个着色器支持不同配置
- 驱动级优化：GPU 驱动可根据常量值进行死代码消除、循环展开、分支预测等优化
- 运行时灵活：可在应用运行时根据设置创建不同特化的管线
- 无运行时开销：数值在管线创建时固化，着色器无须读取

4.3 使用限制

(1) 只在管线创建时设置，一旦管线创建完成，Specialization Constants 的值便固定了，无法在绘制时更改

(2) 支持标量和向量类型 (bool, int, vec2, vec3)，不支持结构体、数组

(3) SPIR-V 必须保留特化信息

(4) 调试难度大，一个着色器对应多个变体，需注意对应关系

4.4 使用场景

- 功能开关，如是否使用法线贴图、SSAO、阴影等
- 配置参数，如最大光源数、粒子数量、纹理采样次数等
- 平台适配，如桌面端和移动端使用不同精度数值或不同逻辑
- 调试模式，如查看法线，查看 GI 等